

Excel Macro Recorder to Dynamic VBA with ChatGPT

Detailed Step-by-Step Guide

Purpose

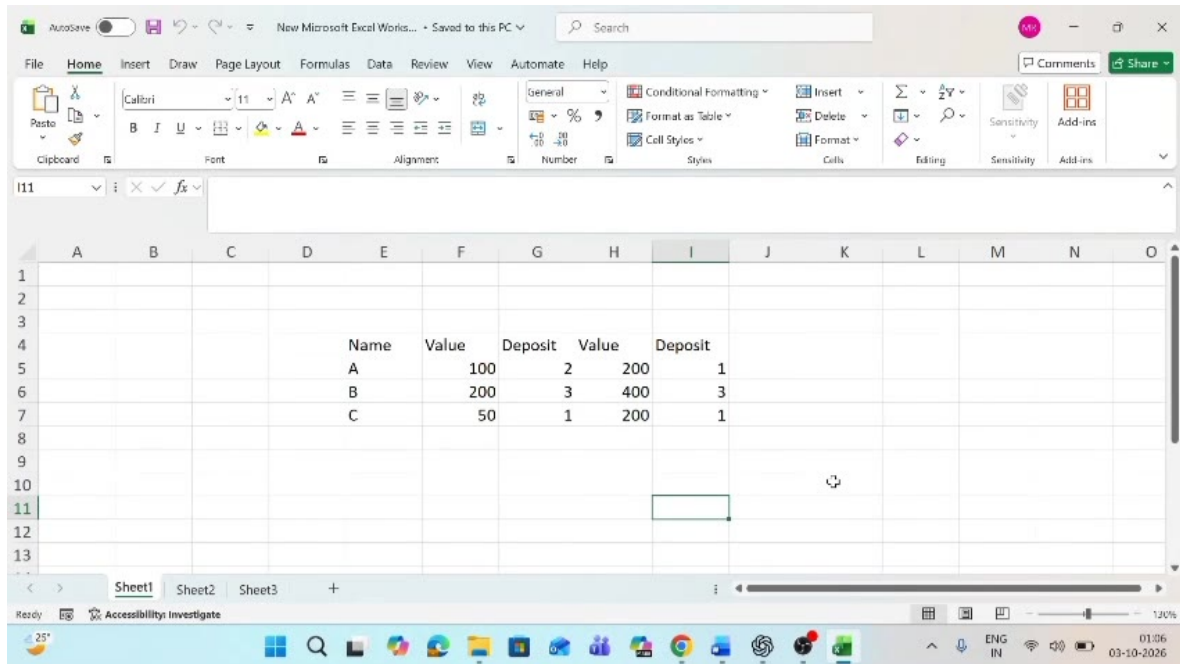
This guide expands the video into a practical tutorial. It explains what to click, why each step is required, what limitation appears in recorded VBA, and how ChatGPT can help rewrite the code to be more dynamic.

Important: Screenshots are visual references from the tutorial video. Excel menu names may differ slightly by version.

Step-by-Step Contents

#	Topic
1	Open Excel and identify your data
2	Enable the Developer tab
3	Start Record Macro
4	Choose where to store the macro
5	Perform the manual Excel task
6	Stop recording
7	Open Visual Basic Editor
8	Open Module1 and inspect the code
9	Understand the fixed-range limitation
10	Test the recorded macro on another worksheet
11	Add new customers/deposits
12	Identify the exact fixed range
13	Copy the recorded VBA
14	Ask ChatGPT to make it dynamic
15	Review the generated VBA
16	Copy the dynamic code
17	Add the new code to Excel
18	Run the dynamic macro
19	Test with new people and deposits
20	Compare the two approaches
21	Safety and best practices

Step 1 — Open Excel and identify your data



Excel screenshot reference — Step 1

What to do

Review the customer/deposit columns and the column where the total will be calculated.

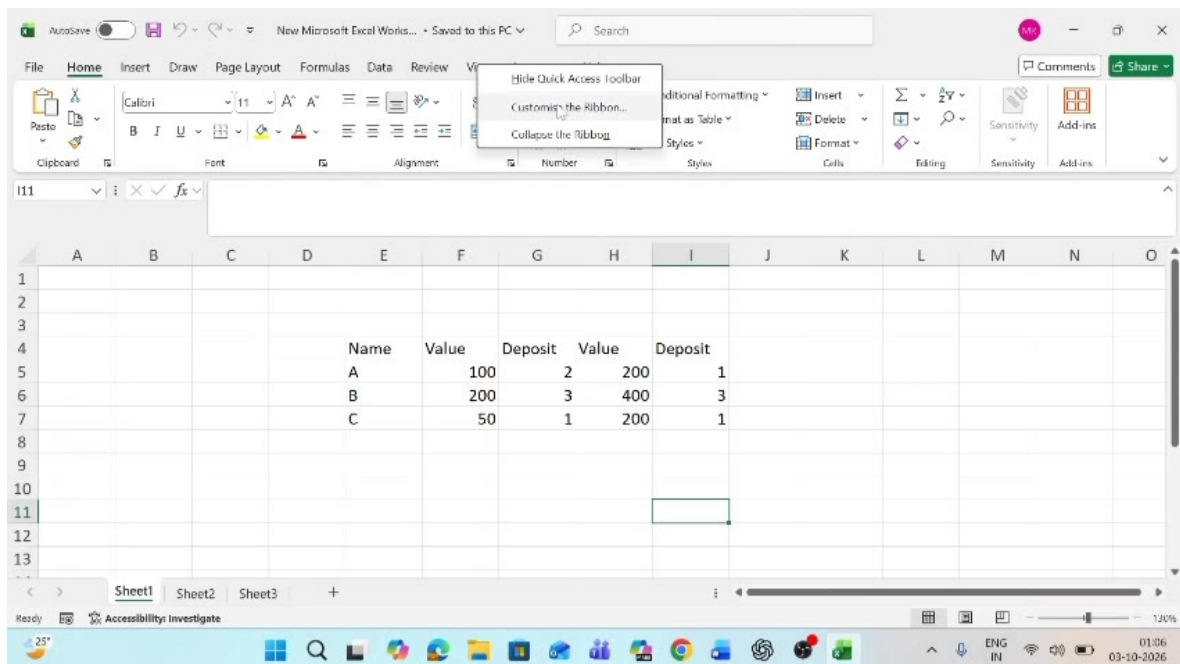
Why this step matters

Knowing the exact manual process helps you understand the VBA generated by the recorder.

Expected result

The source data and result area are clear.

Step 2 — Enable the Developer tab



Excel screenshot reference — Step 2

What to do

Right-click the ribbon > Customize the Ribbon. Tick Developer under Main Tabs and click OK.

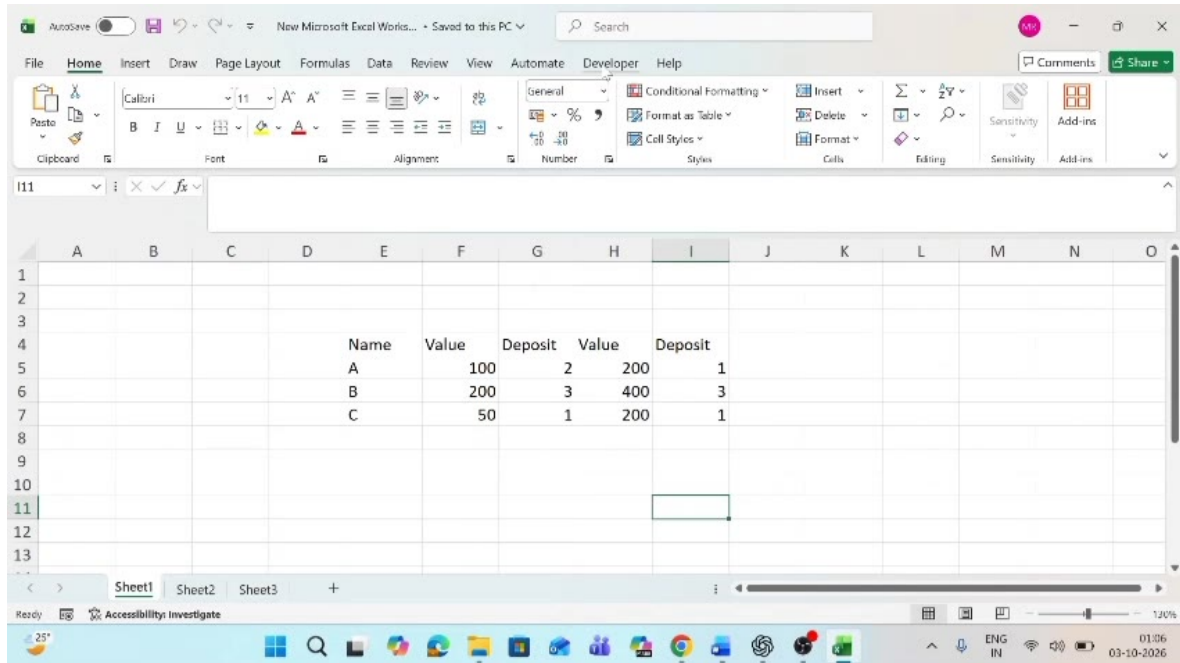
Why this step matters

Developer contains Macro, Visual Basic and other automation tools.

Expected result

Developer appears on the ribbon.

Step 3 — Start Record Macro



Excel screenshot reference — Step 3

What to do

Go to Developer > Record Macro. Enter a meaningful name such as Customer_Deposit.

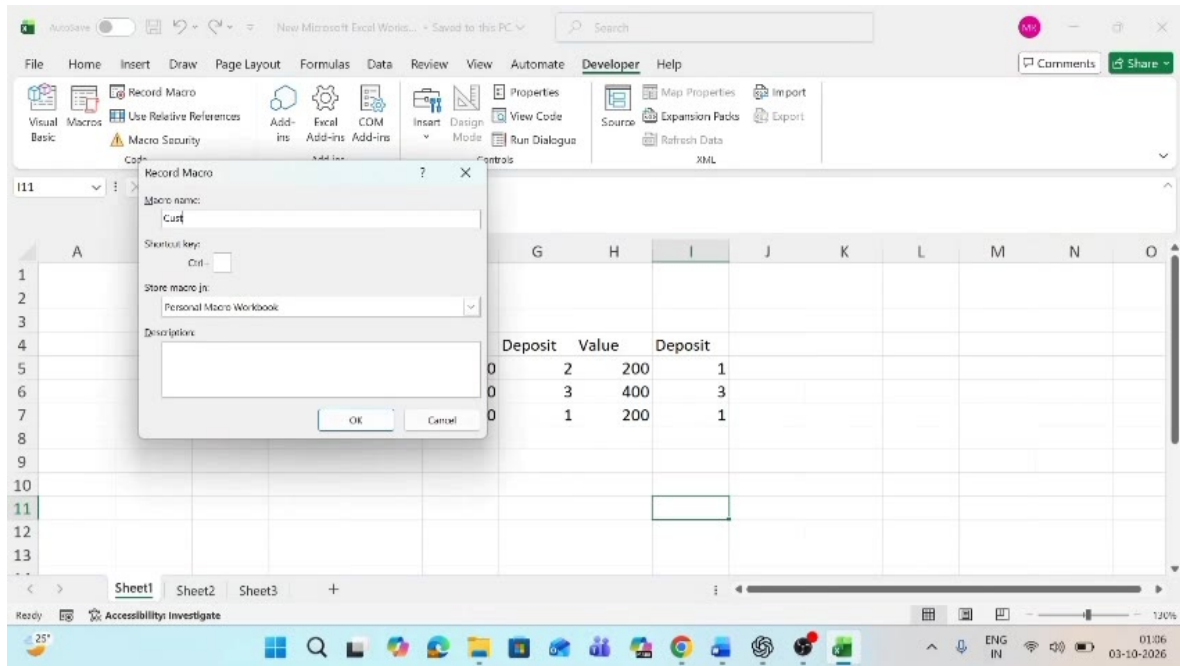
Why this step matters

A clear macro name makes the automation easy to identify and run later.

Expected result

Excel starts recording your actions.

Step 4 — Choose where to store the macro



Excel screenshot reference — Step 4

What to do

In Store macro in, select Personal Macro Workbook if you want to reuse it from other workbooks, then click OK.

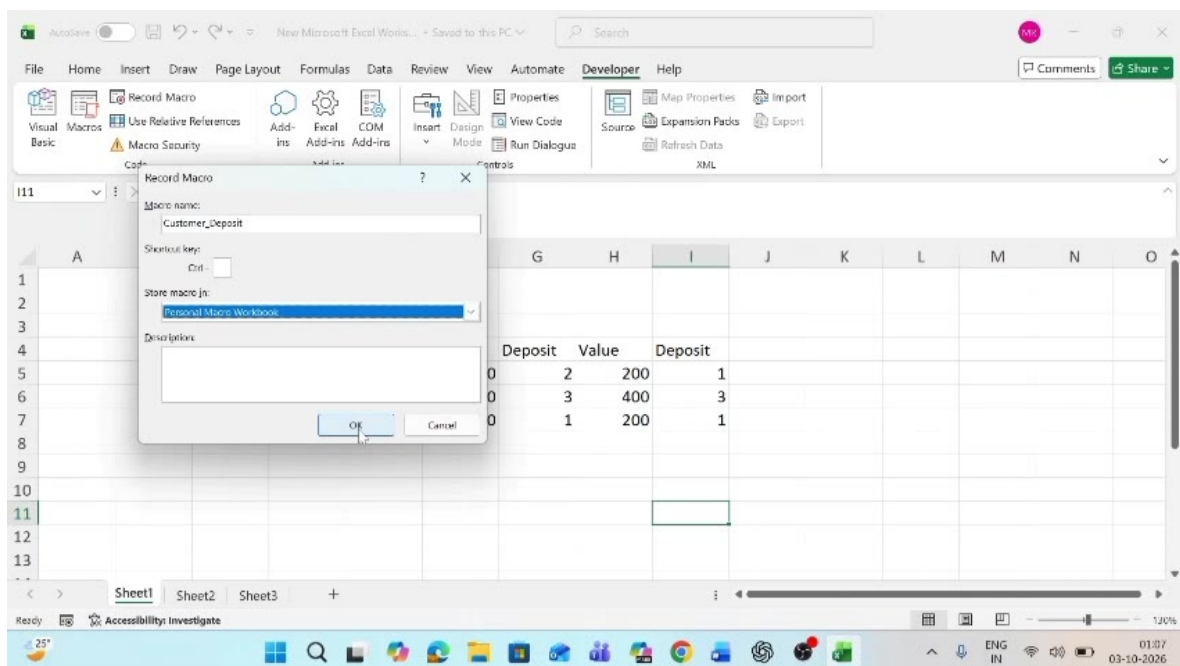
Why this step matters

Personal Macro Workbook is intended for reusable personal macros.

Expected result

Recording is active.

Step 5 — Perform the manual Excel task



Excel screenshot reference — Step 5

What to do

While recording, perform the deposit calculation shown in the video, including the SUM formula.

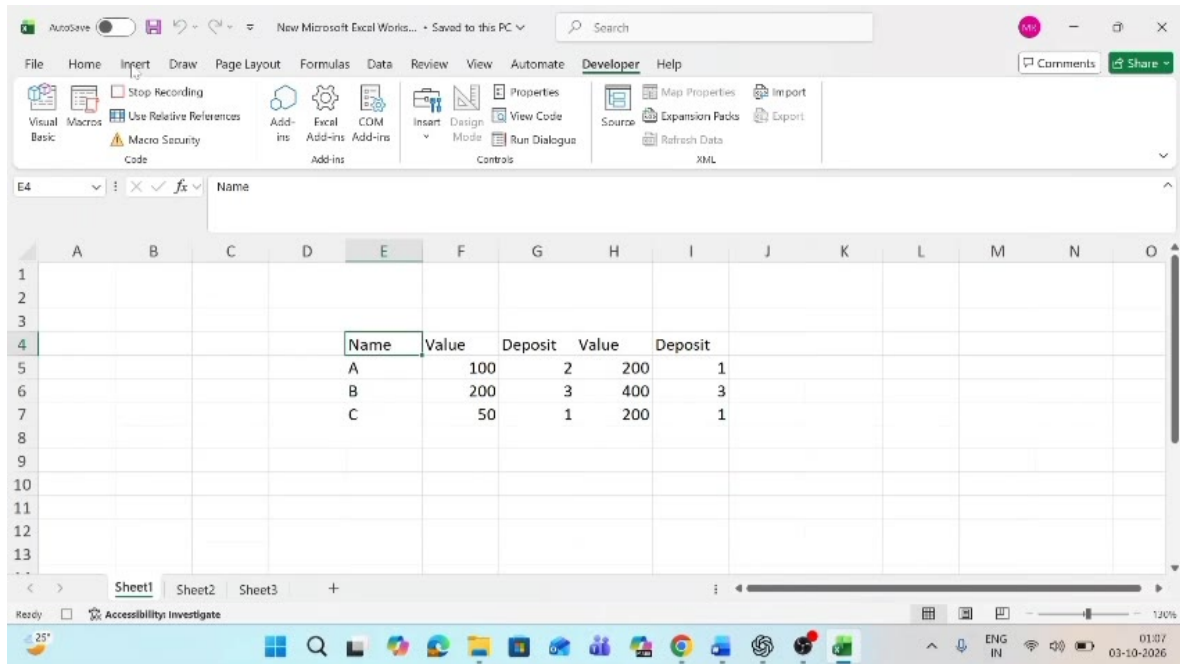
Why this step matters

The Macro Recorder converts your Excel actions into VBA instructions.

Expected result

Your actions are captured.

Step 6 — Stop recording



Excel screenshot reference — Step 6

What to do

Go to Developer > Stop Recording.

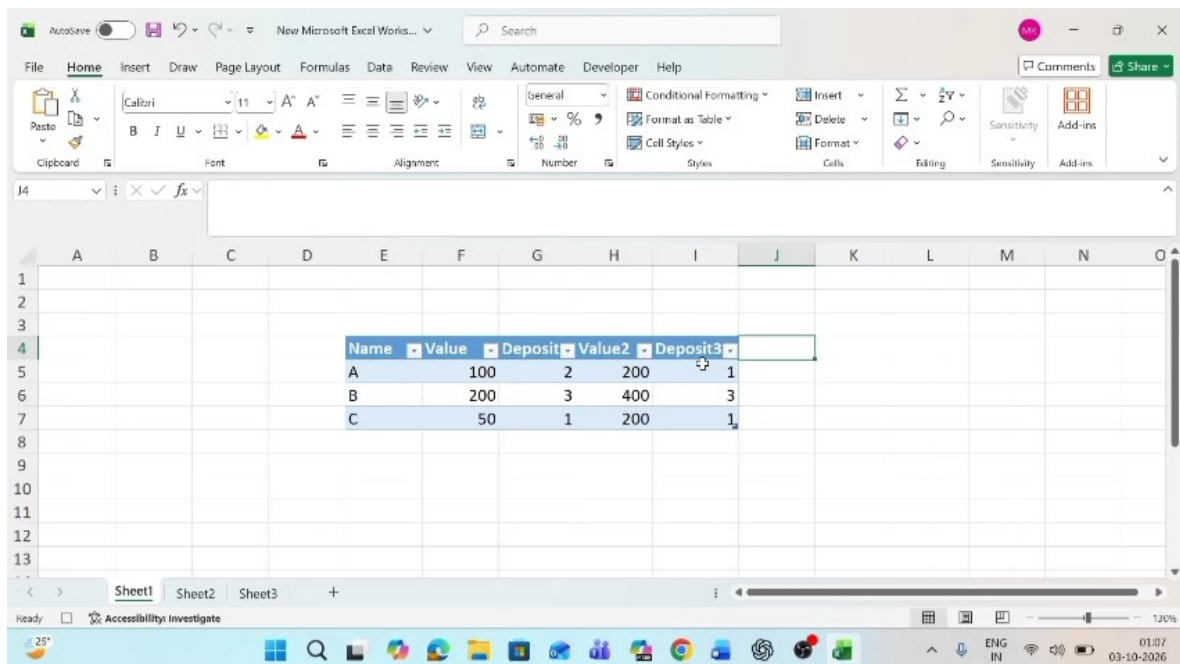
Why this step matters

This saves the recorded VBA procedure.

Expected result

The macro is saved.

Step 7 — Open Visual Basic Editor



Excel screenshot reference — Step 7

What to do

Go to Developer > Visual Basic. Expand the VBA project and Modules.

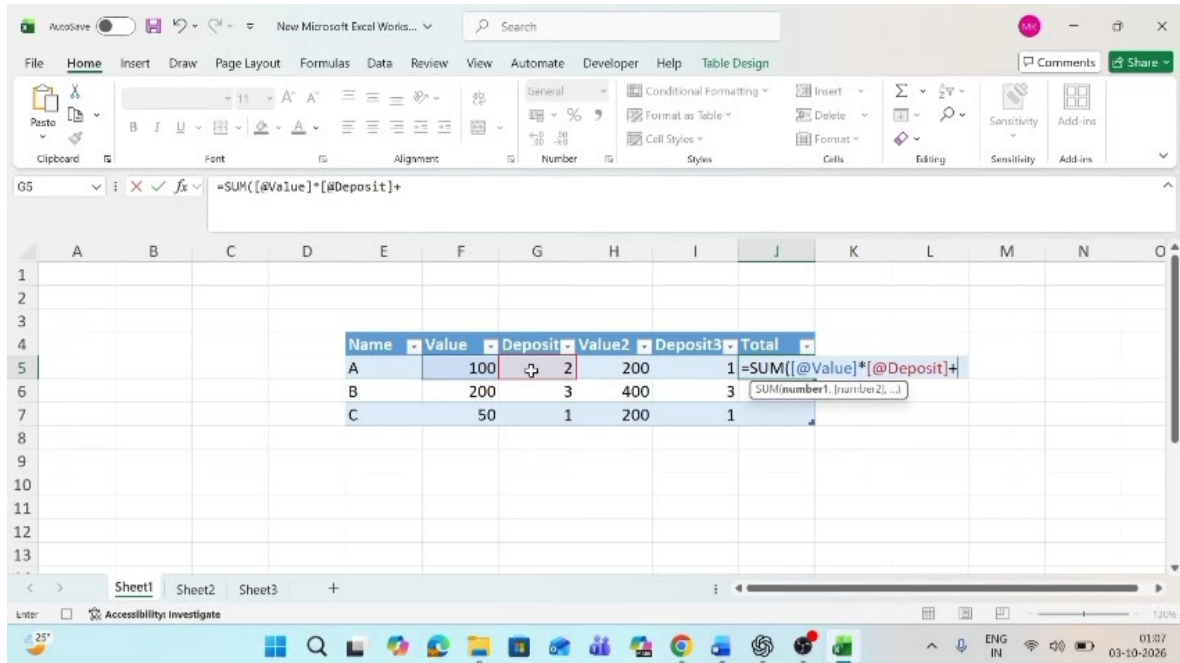
Why this step matters

The editor lets you inspect and modify VBA.

Expected result

The module containing the macro is visible.

Step 8 — Open Module1 and inspect the code



Excel screenshot reference — Step 8

What to do

Double-click Module1 and review the recorded procedure. Look for Range, Select, ActiveCell and FormulaR1C1.

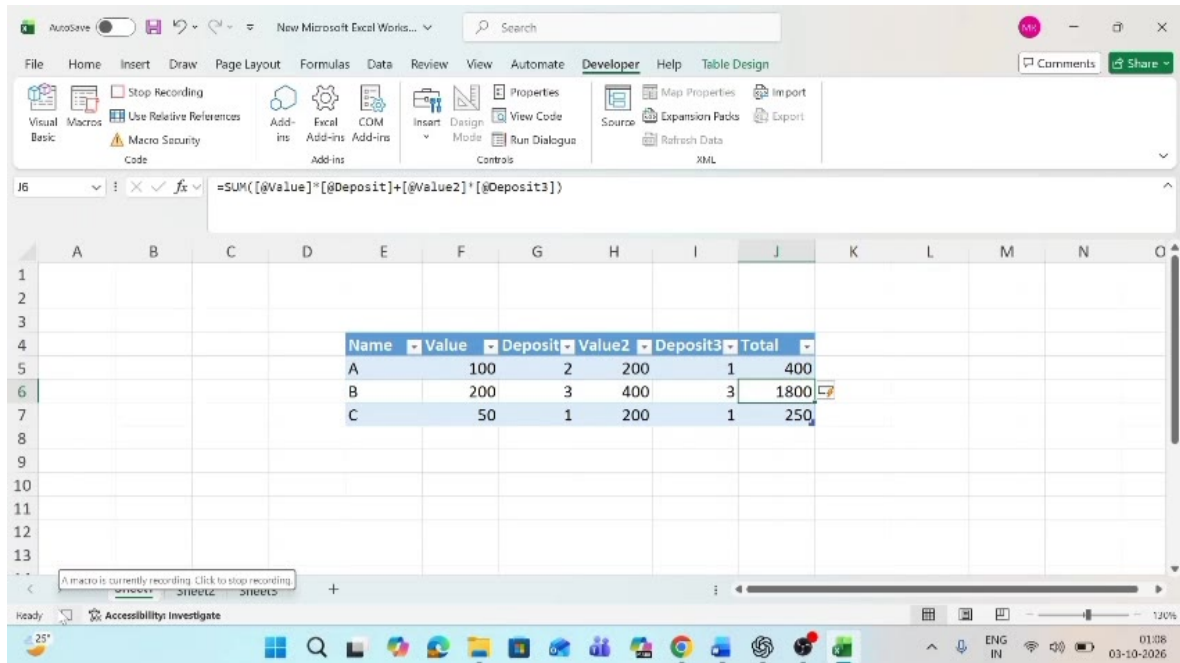
Why this step matters

Recorder code often reflects exact cells and actions used during recording.

Expected result

You can see the generated VBA.

Step 9 — Understand the fixed-range limitation



Excel screenshot reference — Step 9

What to do

Find fixed references such as a range ending at row 7.

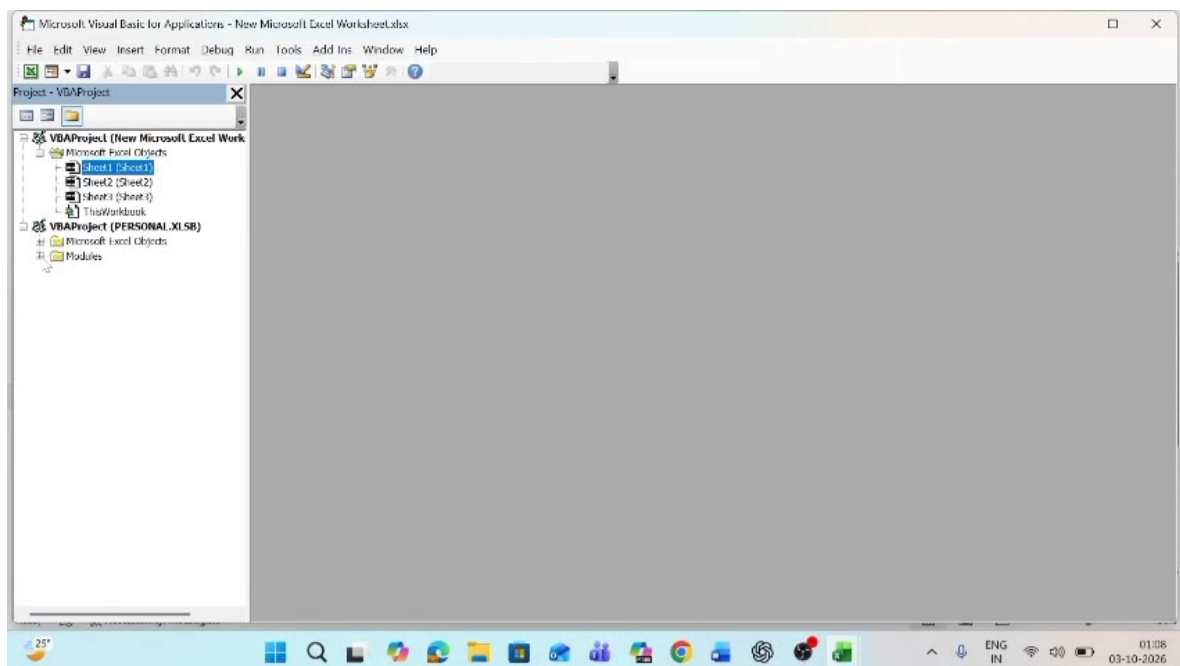
Why this step matters

If new rows are added, the macro may still use the old range.

Expected result

The limitation is visible in the code.

Step 10 — Test the recorded macro on another worksheet



Excel screenshot reference — Step 10

What to do

Create a similar worksheet. Developer > Macros > Customer_Deposit > Run.

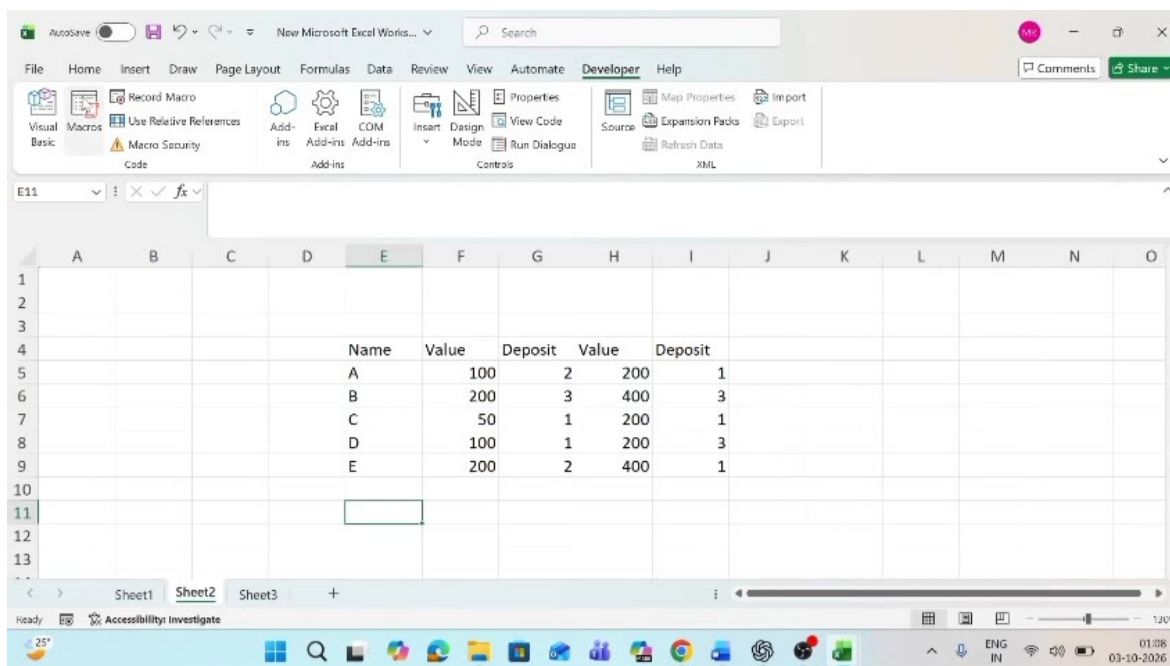
Why this step matters

This shows whether the macro depends on the original range or worksheet.

Expected result

The recorded macro can fail to include new data.

Step 11 — Add new customers/deposits



Name	Value	Deposit	Value	Deposit
A	100	2	200	1
B	200	3	400	3
C	50	1	200	1
D	100	1	200	3
E	200	2	400	1

Excel screenshot reference — Step 11

What to do

Add records beyond the original range and run the macro again.

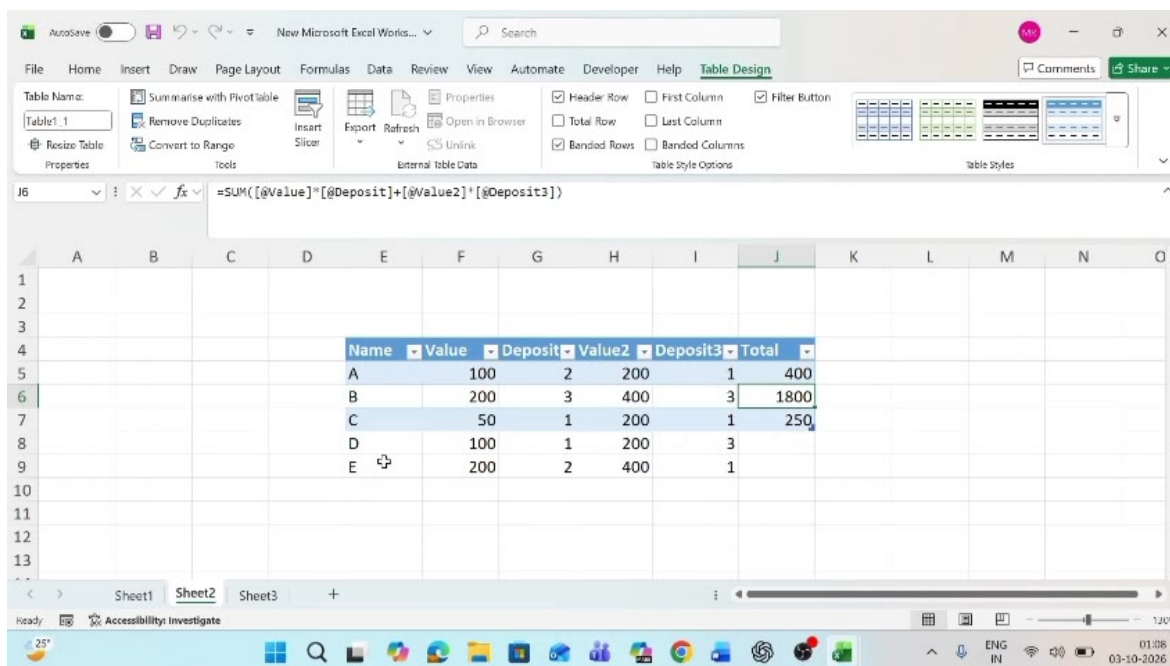
Why this step matters

This reproduces the limitation in a simple way.

Expected result

New records may be missed.

Step 12 — Identify the exact fixed range



Name	Value	Deposit	Value2	Deposit3	Total
A	100	2	200	1	400
B	200	3	400	3	1800
C	50	1	200	1	250
D	100	1	200	3	
E	200	2	400	1	

Excel screenshot reference — Step 12

What to do

Return to Visual Basic and locate the Range statement, such as E4:E7.

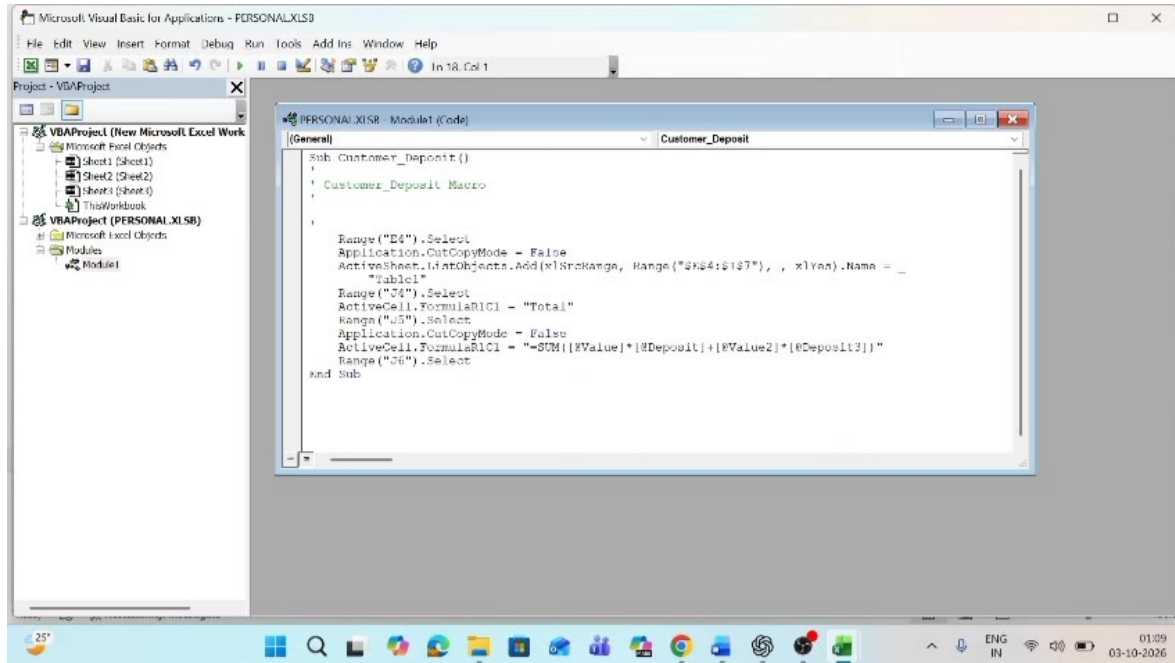
Why this step matters

This is the part that needs to become dynamic.

Expected result

You know what must change.

Step 13 — Copy the recorded VBA



Excel screenshot reference — Step 13

What to do

Copy the complete recorded procedure from the VBA editor.

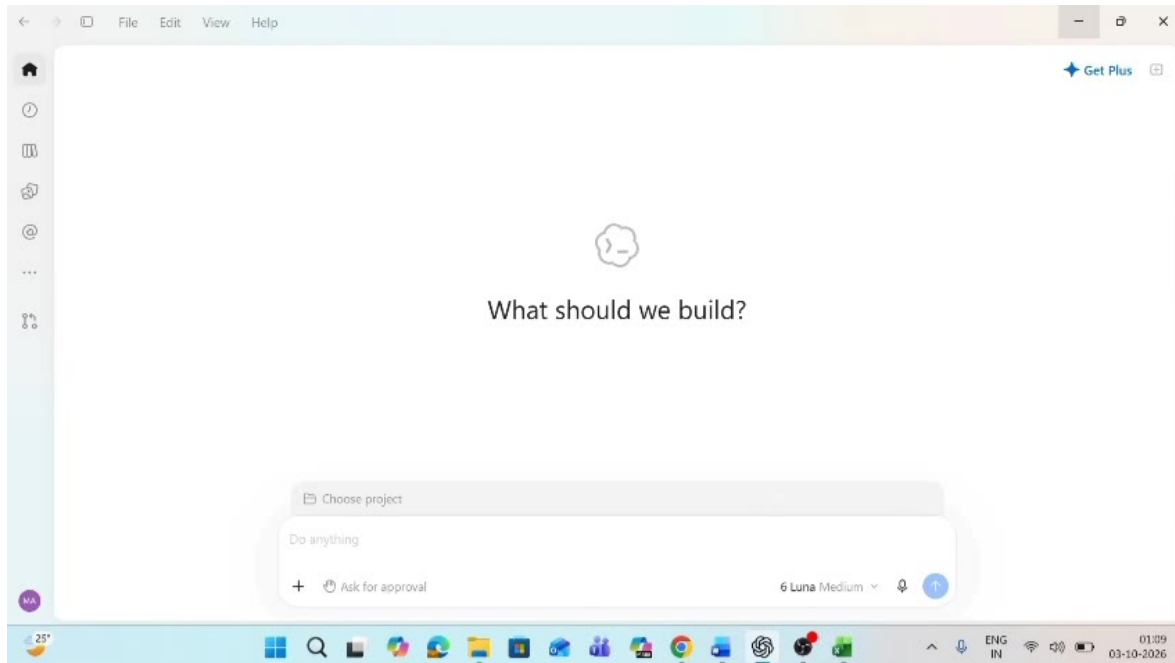
Why this step matters

The recorded procedure is a useful starting point for ChatGPT.

Expected result

The code is ready to paste.

Step 14 — Ask ChatGPT to make it dynamic



Excel screenshot reference — Step 14

What to do

Open ChatGPT and paste the recorded VBA code. Ask for a dynamic version that can run on other worksheets and workbooks without fixed ranges.

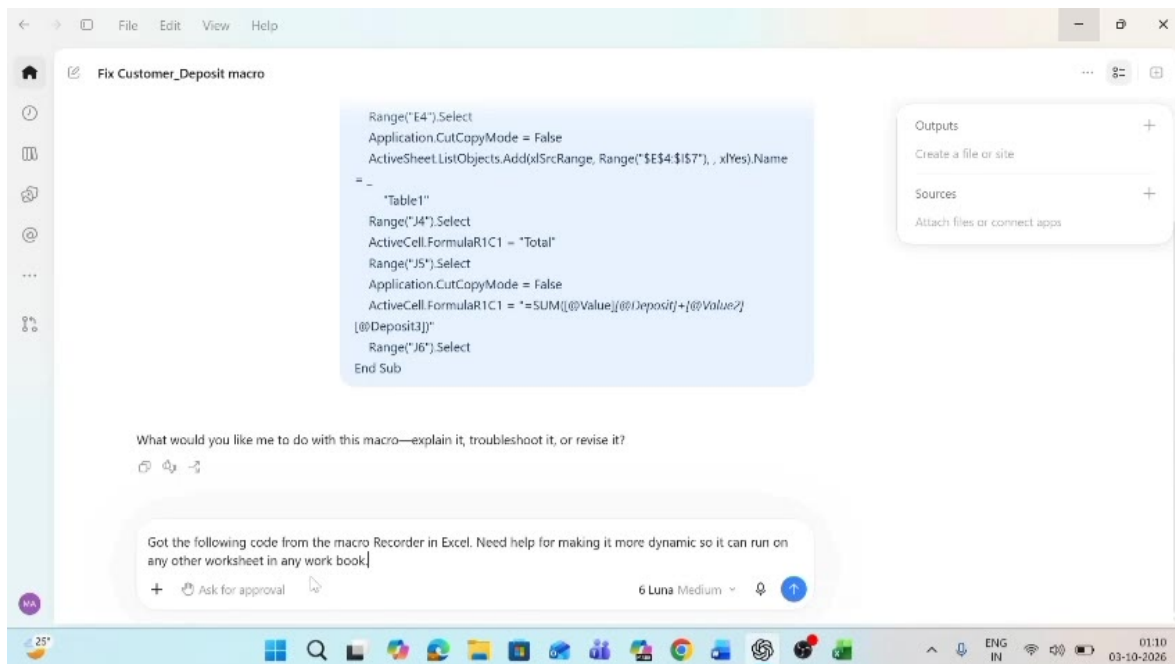
Why this step matters

A precise request helps ChatGPT focus on dynamic range detection and reusable worksheet logic.

Expected result

ChatGPT provides revised VBA.

Step 15 — Review the generated VBA



Excel screenshot reference — Step 15

What to do

Check worksheet references, range detection, formulas, columns and assumptions before using it.

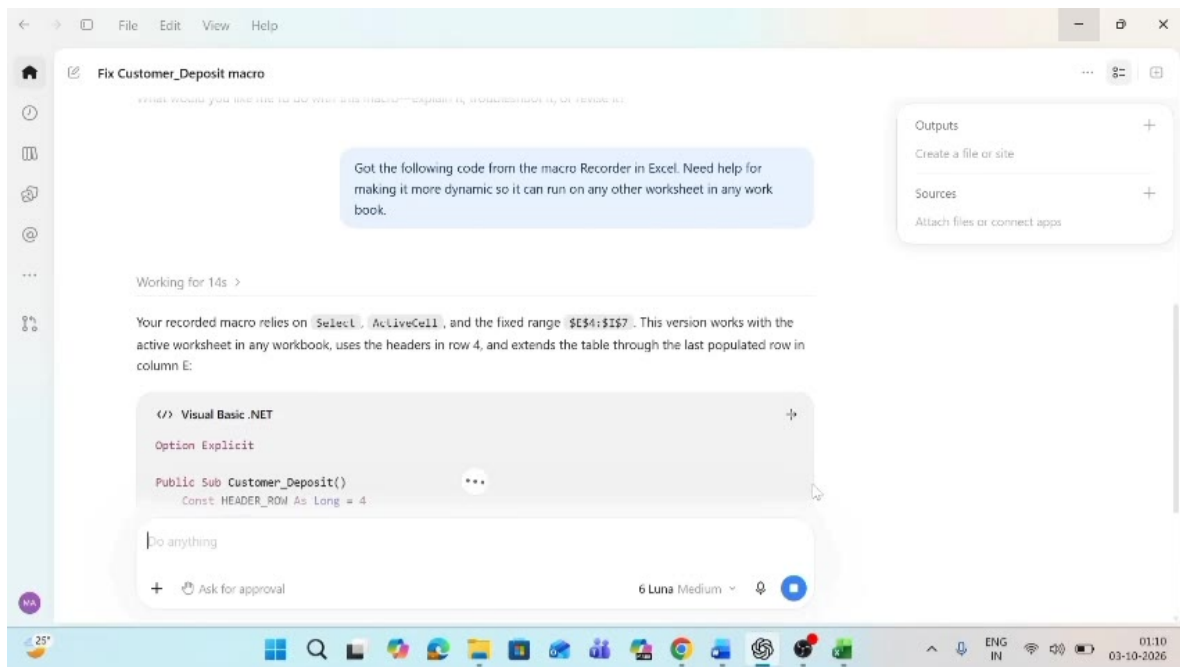
Why this step matters

AI-generated code should be reviewed and tested.

Expected result

You understand what the new code is intended to do.

Step 16 — Copy the dynamic code



Excel screenshot reference — Step 16

What to do

Copy the revised procedure from ChatGPT.

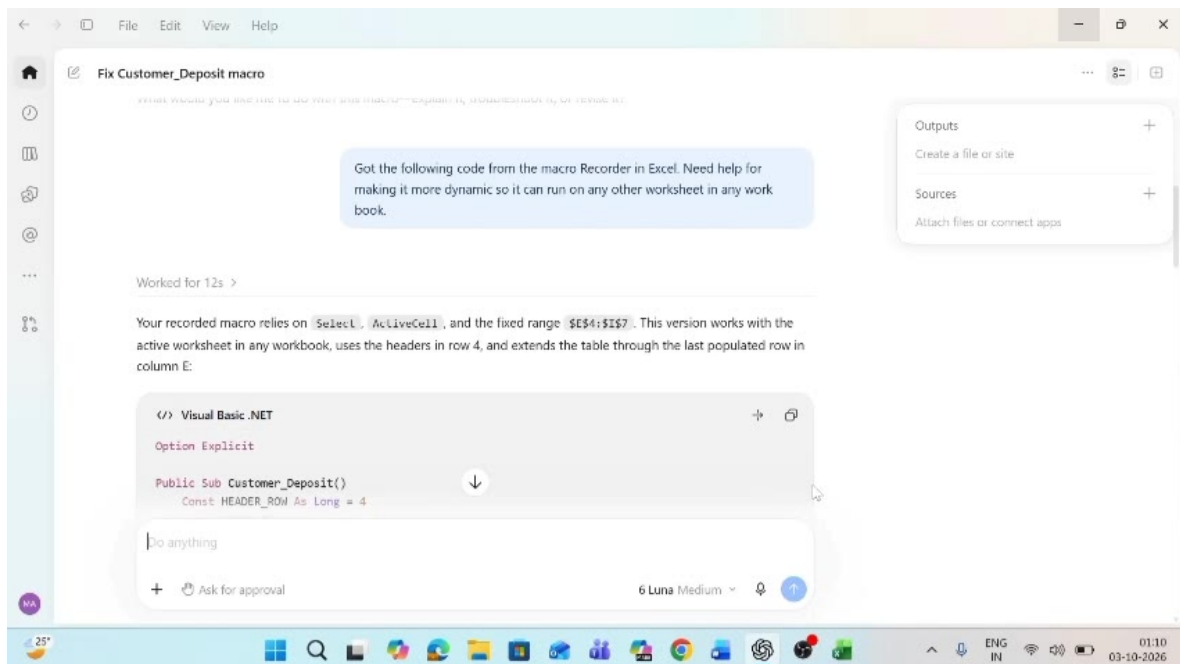
Why this step matters

The new procedure can now be added to Excel.

Expected result

Dynamic code is ready.

Step 17 — Add the new code to Excel



Excel screenshot reference — Step 17

What to do

Return to Visual Basic, paste the procedure into a module and name it Deposit_ChatGPT.

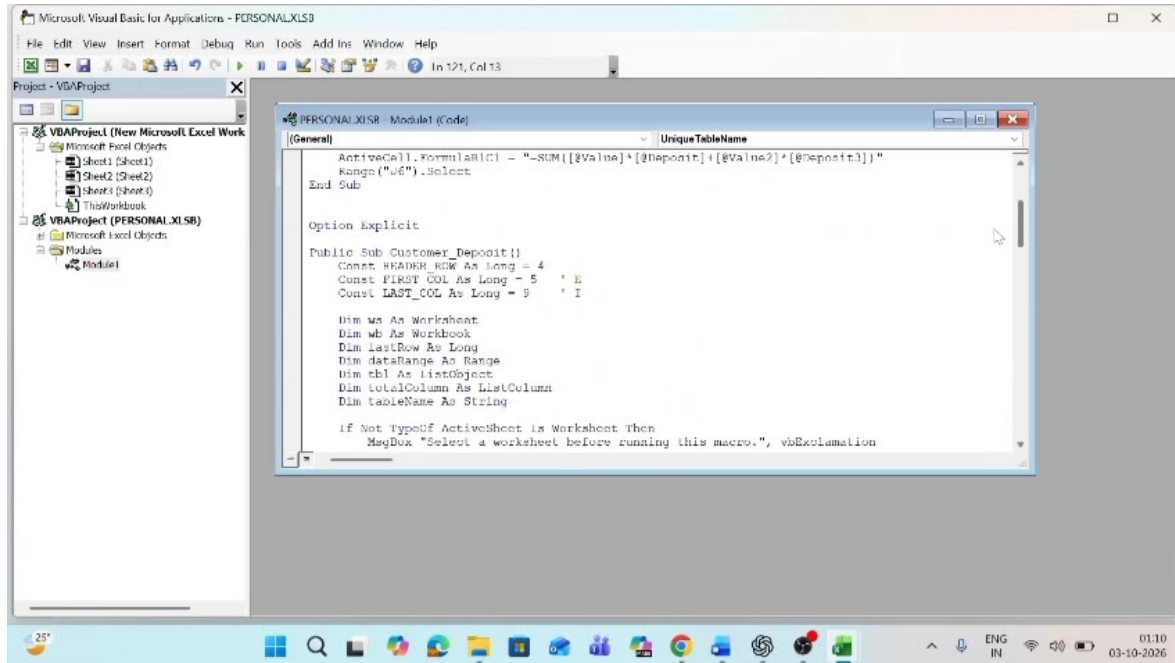
Why this step matters

A separate macro name makes comparison and testing easier.

Expected result

The new macro is available.

Step 18 — Run the dynamic macro



Excel screenshot reference — Step 18

What to do

Return to Excel > Developer > Macros > Deposit_ChatGPT > Run.

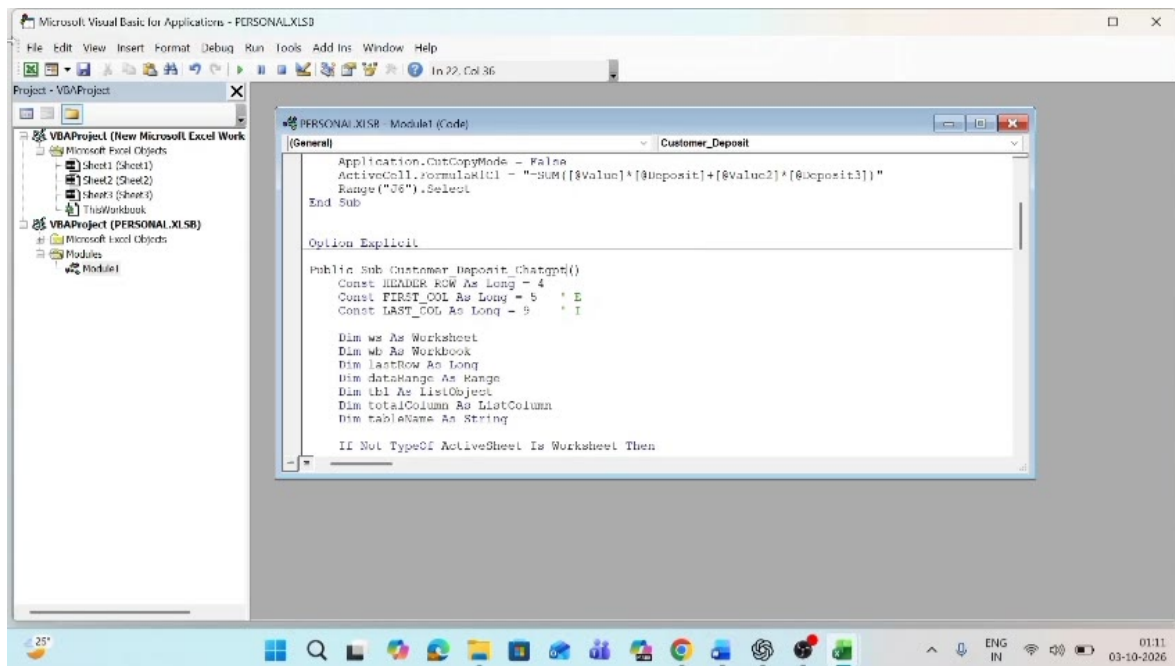
Why this step matters

This tests the revised procedure on the current worksheet.

Expected result

The current data is processed.

Step 19 — Test with new people and deposits



Excel screenshot reference — Step 19

What to do

Add new records and run the dynamic macro again.

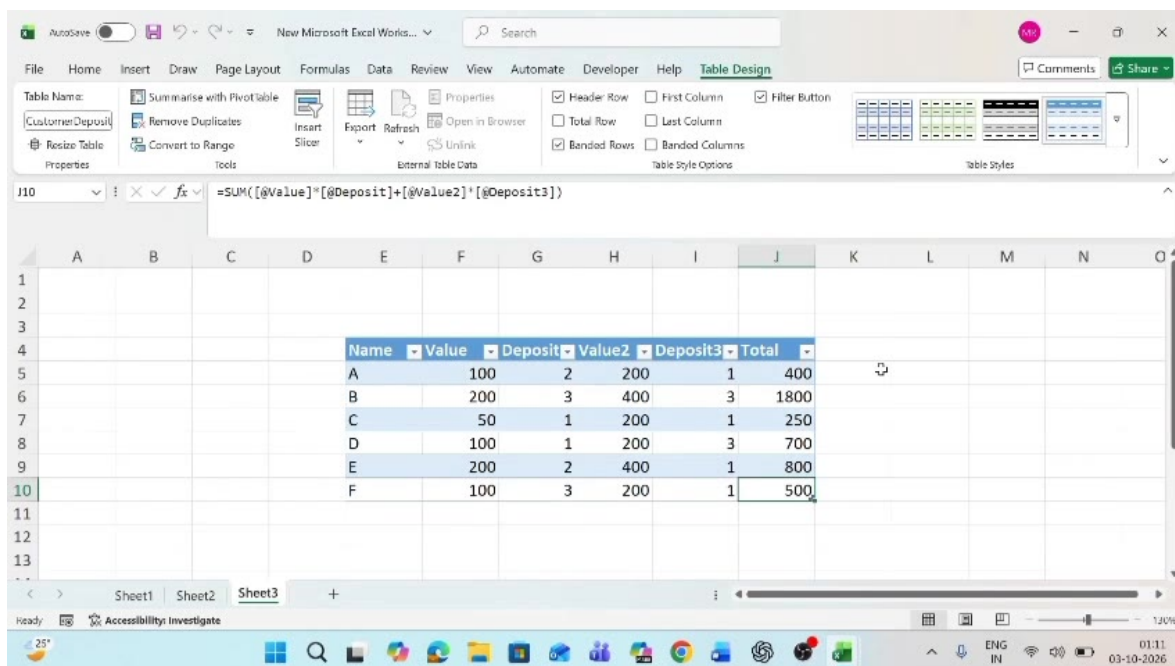
Why this step matters

This is the key test of whether the range adapts to the data.

Expected result

New rows are included.

Step 20 — Compare the two approaches



Excel screenshot reference — Step 20

What to do

Compare the original recorded macro with the dynamic version.

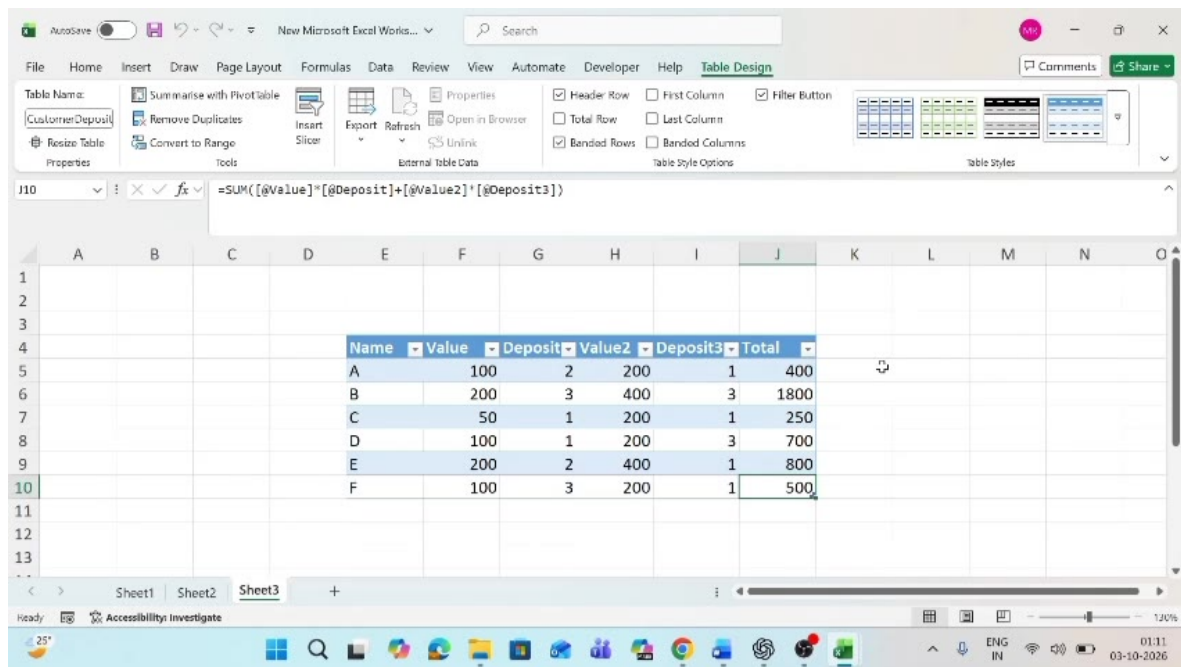
Why this step matters

Macro Recorder is a starting point; VBA editing can make it more reusable.

Expected result

You can see the benefit of dynamic logic.

Step 21 — Safety and best practices



Excel screenshot reference — Step 21

What to do

Test on a copy of important workbooks. Review ranges and results before using VBA on production data.

Why this step matters

VBA can make changes quickly, so backups and testing are important.

Expected result

You can use the automation more safely.

Suggested ChatGPT Prompt

I got the following code from the Macro Recorder in Excel. Please make it more dynamic so it can run on any other worksheet in any workbook. Avoid fixed cell ranges where possible. Detect the actual data range automatically, explain the assumptions about the columns, and provide clean VBA code that does not depend on Select or Activate unless necessary. Please explain the changes and include important error handling.

Before running generated VBA, review worksheet references, column positions, range detection and formulas. Test the macro on a copy of important data first.